Yesterday 7:52 PM

Lab_4_VibeCoding_Student_handout....
PDF

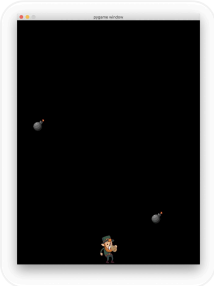
https://github.com/SETAPESU26/63_anvil_dodge/tree/main

Bro, this is your **Lab 4: Vibe Coding** assignment. You've shared your assigned GitHub repository and the lab handout, so the goal is to fix the existing Python game, add the features specified in its `README`, and submit everything in your own repository.

According to the handout, you have **60 minutes**, must work individually, and need to submit three things under `Lab-4` in your lab repository: before-and-after videos, updated code, and an exported PDF or document of your AI chat history. You must not raise a pull request to the original `SETAPESU26` repository. Lab_4_VibeCoding_Student_handout.pdf

What your repository requires

I checked the repository's `README.md`. It's a Python/Pygame game called **Anvil Dodge Repair Lab**, with one deliberate bug and three feature tasks. Read the original README ↗.



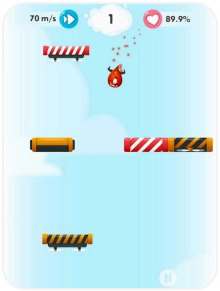
Task 1 — Fix the boundary bug

Prevent the player from moving beyond the left or right edges of the game window.

Task 2 — Dynamic difficulty

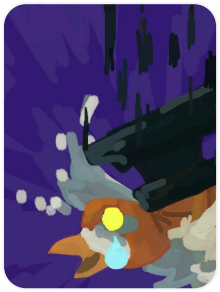
Increase the challenge over time by scaling anvil speed or spawn frequency. Note: the README repeats the boundary-bug description under this task, so this is a reasonable implementation interpretation rather than an explicit specification.





Task 3 — Speed-based warning colours

Use distinct colours for faster anvils so the player can identify more dangerous hazards.



Task 4 — Impact effects

Add dust particles and/or subtle screen shake when anvils hit the ground.

These tasks come from the repository README. SETAPESU26/... +1

Let's get it done in 60 minutes

Lab 4 checklist

- [x] 1. Fork or clone the repository and run the original game
- [ ] 2. Record 10 seconds of gameplay before changes

- 3. Fix the boundary bug and implement all three features
- 4. Test the game and record 10 seconds after changes
- 5. Push to your own GitHub repository
- 6. Save code, both videos and chat history in Lab-4

Use this as a step selector; the items are not saved completion records.

First, open PowerShell

If Git is installed, run:

```powershell
git clone https://github.com/SETAPESU26/63_anvil_dodge.git
cd 63_anvil_dodge
pip install pygame
python main.py
```

Important: Record the original gameplay before modifying any files. Your handout explicitly requires this.

Once the game runs, make a fork or create your own repository before pushing your work. Do not push your changes to the original class repository.

Tell me what you see after running `python main.py` **:** does the game open, or do you get an error? I'll guide you through the actual code changes, prompt by prompt, so you can finish the lab and understand what you're submitting.

```
oads> git clone https://github.com/SETAPESU26/63_anvil_dodge.git
Cloning into '63_anvil_dodge'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 15 (delta 4), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), 7.51 KiB | 349.00 KiB/s, done.
Resolving deltas: 100% (4/4), done.
PS C:\Users\nadhe\downloads> cd 63_anvil_dodge
PS C:\Users\nadhe\downloads\63_anvil_dodge> pip install pygame
Collecting pygame
  Downloading pygame-2.6.1-cp312-cp312-win_amd64.whl.metadata (13 kB)
Downloading pygame-2.6.1-cp312-cp312-win_amd64.whl (10.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.6/10.6 MB 12.7 MB/s eta 0:00:00
Installing collected packages: pygame
Successfully installed pygame-2.6.1

[notice] A new release of pip is available: 25.0.1 -> 26.2.1
[notice] To update, run: C:\Users\nadhe\AppData\Local\Programs\Python\Python312\python.exe -
m pip install --upgrade pip
PS C:\Users\nadhe\downloads\63_anvil_dodge> python main.py
Traceback (most recent call last):
  File "C:\Users\nadhe\downloads\63_anvil_dodge\main.py", line 1, in <module>
    import pygame
```

ModuleNotFoundError: No module named 'pygame'
PS C:\Users\nadhe\downloads\63_anvil_dodge>

Bro, pygame installed successfully, but your `python main.py` is using a Python environment that cannot find it. This is usually a Python/pip mismatch on Windows. Let's fix it without reinstalling everything.

Step 1: Run these commands

You're already in the correct project directory. Copy and paste these commands into PowerShell, one at a time:

powershell

```
python --version
python -m pip --version
python -m pip install pygame
python -c "import pygame; print(pygame.version.ver)"
```

The important command is `python -m pip install pygame`. It installs pygame into the environment associated with the exact `python` command you're using to run the game.

Step 2: Run your game again

powershell

```
python main.py
```

If it works, the game should open. **Record your 10-second video before making any changes**, as required by your lab handout.

If you still get the same error

Run this diagnostic command:

powershell

```
where.exe python
where.exe pip
python -c "import sys; print(sys.executable)"
pip show pygame
python -m pip show pygame
```

Paste the output here. We'll identify the mismatch instead of randomly changing your Python installation.

ads\63_anvil_dodge> python --version
Python 3.14.6
PS C:\Users\nadhe\downloads\63_anvil_dodge> python -m pip --version
C:\msys64\ucrt64\bin\python.exe: No module named pip
PS C:\Users\nadhe\downloads\63_anvil_dodge> python -m pip install pygame
C:\msys64\ucrt64\bin\python.exe: No module named pip
PS C:\Users\nadhe\downloads\63_anvil_dodge> python -c "import pygame;
print(pygame.version.ver)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import pygame; print(pygame.version.ver)
    ^^^^^^^^^^^^^^

ModuleNotFoundError: No module named 'pygame'
PS C:\Users\nadhe\downloads\63_anvil_dodge>

Bro, found the issue! Your terminal is using **MSYS2's Python 3.14.6**, while `pip install pygame` installed pygame for a different Python installation (Python 3.12). That's why Python cannot import it.

Let's use the Windows Python installation that already has pip and pygame. No need to reinstall Python.

Step 1 — Find your Windows Python

Copy and run this in PowerShell:

```powershell
py -0p
```

This lists the Python versions installed on Windows and their paths.

Step 2 — Try Python 3.12 directly

Your earlier output showed this Python 3.12 installation:

`C:\Users\nadhe\AppData\Local\Programs\Python\Python312\python.exe`

Run these commands:

```powershell
```

```
& "C:\Users\nadhe\AppData\Local\Programs\Python\Python312\python.exe" -c "import pygame; print(pygame.version.ver)"
```

If it prints a pygame version, run the game using that same interpreter:

powershell

```
& "C:\Users\nadhe\AppData\Local\Programs\Python\Python312\python.exe" main.py
```

Step 3 — If that Python cannot import pygame

Install pygame into Python 3.12 explicitly:

powershell

```
& "C:\Users\nadhe\AppData\Local\Programs\Python\Python312\python.exe" -m pip install pygame
```

Then launch the game again using the Step 2 command.

Why this works: `pip` and `python` must refer to the same installation. Your current `python` points to `C:\msys64\ucrt64\bin\python.exe`, which doesn't have pip.

Once the game launches, record the original 10-second video. Then we'll tackle the broken code and all three requested features.

Today 12:19 PM

next what should i do

Okay bro, let's finish Lab 4 step by step. Don't jump into editing the code yet — first we need the original game running and your before-video recorded.

Step 1: Launch the game

In your current PowerShell terminal, run:

```powershell
& "C:\Users\nadhe\AppData\Local\Programs\Python\Python312\python.exe" main.py
```

- If the game opens, move the player around and check that the game works.
- If you get an error, send me the complete error message.

Step 2: Record the original gameplay

Once the game opens:

1. Press `Win + G` to open Xbox Game Bar.
2. Start recording with `Win + Alt + R`.
3. Play for at least 10 seconds.
4. Stop recording with `Win + Alt + R`.

Save the video somewhere easy to find, such as your project folder, named `before.mp4`.

Step 3: Check your project files

Stop the game with `Esc` or close its window. Then run:

```powershell
dir
```

We need to identify the Python files and README before making changes.

Step 4: Fix the game and add the features

Your assignment requires these changes:

- Fix the player moving beyond the screen boundaries.
- Implement increasing difficulty over time.
- Change anvil colours according to speed.
- Add impact effects, such as dust particles or screen shake.

We'll use ChatGPT to make and test each change, then record the final gameplay.

Step 5: Push everything to your own GitHub repository

After testing, we'll upload the updated code, both videos, and your exported AI chat history inside the `Lab-4` folder of your lab repository.

For now, do Step 1 and tell me what happens: does the game open, or do you see another error?

i recorded the video now

Nice bro! 🔥 The original gameplay video is done. Now let's fix the game and add the required features.

Step 1 — Inspect your project files

In your existing PowerShell terminal, run:

```powershell
dir
```

Then run:

```powershell
Get-Content README.md
```

This lets us verify the exact requirements and identify the files we need to modify.

Step 2 — Show me the game code

Run:

```powershell
Get-Content main.py
```

Copy and paste the output here.

If `main.py` is very long, use:

powershell

```powershell
Get-Content main.py -TotalCount 200
```

Step 3 — What we'll implement

We'll work through these changes:

1. **Boundary fix:** Keep the player inside the game window.
2. **Dynamic difficulty:** Make the game progressively harder.
3. **Warning colours:** Change anvil colours based on their speed.
4. **Impact effects:** Add dust particles or screen shake when anvils hit the ground.

We'll test the game after the changes and record your second 10-second video.

Your next action: Run `Get-Content main.py` and send me the output. I'll help you modify the actual code rather than guessing how your game is structured.

This project is a modular survival dodger game using **Pygame**. It introduces students to falling hazard mechanics, boundary management, collision detection, and survival time tracking within a clean, object-oriented codebase.

What's Provided

A working Anvil Dodge game with:

- A player character that can move left and right across the ground line using keyboard inputs
- Heavy anvils spawned continuously from random horizontal positions falling toward the ground

- Collision detection when an anvil hits the player, triggering game over
- Real-time survival time tracking and a Game Over overlay with restart functionality

It has **one deliberate bug** and **three optional features** left as tasks to implement. You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
pip install pygame
```

3. Run the game:

```
python main.py
```

Controls: Left / A to move left, Right / D to move right, R to reset after Game Over.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the player off-screen boundary bug

The player character can walk past the left and right screen edges into hidden space where falling anvils cannot hit them, exploiting an infinite survival time. Constrain the player so movement is strictly clamped within the visible boundaries of the display window.

Task 2: Implement dynamic difficulty scaling

The player character can walk past the left and right screen edges into hidden space where falling anvils cannot hit them, exploiting an infinite survival time. Constrain the player so movement is strictly clamped within the visible boundaries of the display window.

Task 3: Implement speed-based anvil warning tints

All falling hazards currently share the same grey appearance regardless of velocity. Apply visual hazard tiers to falling anvils by tinting higher-velocity anvils with distinct hot warning colors to signal urgent threats to the player.

Task 4: Implement ground impact FX & screen shake

Anvils reaching the floor vanish silently without physical presence. Add kinetic feedback on impactâ€”such as localized dust cloud particles or a subtle camera shake whenever an anvil smashes into the ground line.

Expected Behavior

- The player cannot move beyond the visible left and right edges of the screen
- Anvils fall continuously from randomized X coordinates and clean up after passing below the screen
- Touching any falling anvil immediately triggers the Game Over screen and stops time tracking
- Pressing R after losing resets the player, clears all falling anvils, and restarts the survival timer

---

Folder Structure

```
anvil_dodge/
â”œâ”€â”€ game/
â”‚   â”œâ”€â”€ anvil.py
â”‚   â”œâ”€â”€ game_engine.py
â”‚   â””â”€â”€ player.py
â”œâ”€â”€ main.py
â””â”€â”€ README.md
```

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

PS C:\Users\nadhe\Downloads\63_anvil_dodge> Get-Content main.py

```
import pygame
from game.game_engine import GameEngine

WIDTH, HEIGHT = 700, 500
FPS = 60


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Anvil Dodge - Pygame Edition")
    clock = pygame.time.Clock()

    engine = GameEngine(WIDTH, HEIGHT)
```

```
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.update()
        engine.render(screen)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
```

Perfect bro. 🔥 We've confirmed that `main.py` only initializes Pygame and runs the game loop. The actual game logic lives in the `game` folder, so that's where we'll make most of the changes.

One important detail: your README repeats the boundary-bug description under Task 2. We'll implement **actual dynamic difficulty scaling** as well, so all three optional features are covered.

Step 1 — Inspect the three game files

Copy and paste these commands into your existing PowerShell terminal:

1. Player movement and boundaries

powershell

```
Get-Content game/player.py
```

Copy command

2. Anvil movement and appearance

powershell

```
Get-Content game/anvil.py
```

Copy command

3. Game logic, collisions and survival timer

powershell

```
Get-Content game/game_engine.py
```

Copy command

Step 2 — Send me the output

Paste all three files here. If the output is too long, send them in separate messages.

I'll help you update the existing code while preserving its structure and controls.

What we'll implement

- **Boundary fix:** Clamp player movement to the screen edges.
- **Dynamic difficulty:** Gradually increase anvil speed or spawn frequency.
- **Warning tints:** Use grey for slower anvils and hotter colours for faster ones.
- **Impact effects:** Add dust particles and subtle screen shake when anvils hit the ground.

We'll then run the game, test each feature, record your after-video, and prepare the submission.

Don't overwrite any files yet. Let's inspect the original implementation first so we can make the right changes in one pass.

anvil.py
Code

game_engine.py
Code

player.py
Code

Analyzed

Bro, I've reviewed all three files. We can implement the features directly in your existing code without changing the project structure. Your original game already has collision detection, survival-time tracking, and restart functionality, so we'll preserve those.

Here's what I found in your code:

- `player.py` : `move_left()` and `move_right()` change the player's position without checking screen boundaries. That's the deliberate bug.
- `anvil.py` : Every anvil uses fixed grey colours, and its speed is randomly selected between 4.5 and 7.0.
- `game_engine.py` : The game spawns anvils every 700 milliseconds, but the difficulty never increases and ground impacts have no effects.

One small correction to the README: Task 2 repeats the boundary bug description. We'll implement genuine dynamic difficulty scaling as well as the other two optional features.

Step 1 — Download the updated files

I've prepared all three updated Python files for you.

Anvil Dodge — Updated Code

ZIP archive · 3 Python files

Download the updated game files ↗

Step 2 — Replace the existing files

1. Download and extract the ZIP.
2. Open the extracted `game` folder.
3. Copy these three files into your original project at `C:\Users\nadhe\Downloads\63_anvil_dodge\game\`, replacing the existing files.

Your project should look like this:

Plain text

```
63_anvil_dodge/
├── game/
│   ├── anvil.py
│   ├── game_engine.py
│   └── player.py
├── main.py
└── README.md
```

Leave `main.py` unchanged.

Step 3 — Run and test the game

In PowerShell, run:

```powershell
& "C:\Users\nadhe\AppData\Local\Programs\Python\Python312\python.exe" main.py
```

Check each feature:

- [ ] Player cannot move off either screen edge.
- [ ] Anvils become faster and spawn more frequently over time.
- [ ] Faster anvils change from grey to orange or red.
- [ ] Anvils hitting the ground create dust particles and screen shake.
- [ ] Collision triggers Game Over, and pressing `R` restarts the game.

If you encounter an error, send me the full traceback before changing anything else.

Step 4 — Record the after-video

Once everything works, record another 10-second video. Try to capture the warning colours and ground-impact effects. If you can, keep playing longer before recording so the increased difficulty is visible.

Step 5 — Finish your submission

You'll need your original video, the final video, updated code, and the complete chat history or shareable chat link, following the exact submission instructions in your README.

One important thing: These changes are implemented in the prepared files, but we haven't verified them running on your computer yet. Test first, then we'll handle GitHub and the final submission.

‹/›

how and where do i update it in the fork repo or where

Bro, update the code in **your own fork of the repository**, not the original `SETAPESU26` repository. Since you've already cloned the project to your laptop, the easiest workflow is to update the local files, test them, and then push the changes to your fork. 🔥

Step 1 — Fork the original repository

Open the original repository:

Open the Anvil Dodge repository ↗

1. Sign in to GitHub.
2. Click **Fork** in the top-right corner.
3. Create the fork under your personal GitHub account.

Your fork should have a URL similar to:

```
https://github.com/YOUR_USERNAME/63_anvil_dodge
```

Replace `YOUR_USERNAME` with your GitHub username.

Step 2 — Update the code on your laptop

You've already cloned the repository here:

```
C:\Users\nadhe\Downloads\63_anvil_dodge
```

You don't need to clone it again.

Your project folder

`Downloads\63_anvil_dodge\game\`

Download and extract the updated files I prepared:

Download updated Python files ↗

Replace the three existing files inside your local `game` folder:

- `player.py`
- `anvil.py`
- `game_engine.py`

Keep `main.py` and `README.md` unchanged for now.

Step 3 — Test your changes

In PowerShell:

powershell

```powershell
cd C:\Users\nadhe\Downloads\63_anvil_dodge
& "C:\Users\nadhe\AppData\Local\Programs\Python\Python312\python.exe" main.py
```

Confirm that the boundaries, difficulty scaling, warning colours, and impact effects work.

Step 4 — Connect your local project to your fork

After creating the fork, replace `YOUR_USERNAME` with your actual GitHub username and run:

```powershell
git remote -v
git remote set-url origin https://github.com/YOUR_USERNAME/63_anvil_dodge.git
git add game/player.py game/anvil.py game/game_engine.py
git commit -m "Fix boundaries and add gameplay features"
git push -u origin main
```

This sends your changes to **your fork**, not the original repository.

If Git asks you to authenticate, complete the GitHub sign-in flow.

Step 5 — Where to put the lab submission

Your handout says to submit the videos, updated code, and chat history in the `Lab-4` folder of your lab repository.

Keep that submission separate from the game repository unless your lab repository specifically requires otherwise.

One question before you push: Have you already clicked **Fork** on GitHub, or have you only cloned the repository onto your laptop?

PDF **Lab_4_VibeCoding_Student_handout(...**
PDF

Bro, I have the Lab 4 handout again. The key requirement is that you must **push the updated game code to your own GitHub repository** and submit the two videos, updated code, and exported chat history under `Lab-4` in your lab repository. You must not raise a PR to `SETAPESU26` .

Lab_4_VibeCoding_Student_handout.pdf

Let's continue from where we stopped. **Have you created your GitHub fork yet?**

- **A. Yes, I've created the fork.**
- **B. No, I've only cloned the repository on my laptop.**

Choose A or B, and I'll give you the exact next commands.

i cloned it and updated the files

Perfect, bro! You've cloned the repository and updated the files locally. Now we need to **test your changes, create your own GitHub fork, and push the code there**. Don't push to the original class repository.

Step 1 — Test the updated game

In PowerShell, run:

```powershell
cd C:\Users\nadhe\Downloads\63_anvil_dodge
& "C:\Users\nadhe\AppData\Local\Programs\Python\Python312\python.exe" main.py
```

Check that the game opens and verify the boundary fix, increasing difficulty, warning colours, and impact effects.

If you get an error, send me the full error message before proceeding.

Step 2 — Create your fork on GitHub

1. Open the original repository: 63_anvil_dodge on GitHub ↗.
2. Click **Fork**.
3. Choose your personal GitHub account and create the fork.

Step 3 — Push your updated files

Once your fork exists, return to PowerShell and run:

```powershell
git status
git add game/player.py game/anvil.py game/game_engine.py
git commit -m "Fix boundaries and add gameplay features"
git remote add myfork https://github.com/YOUR_USERNAME/63_anvil_dodge.git
git push -u myfork HEAD
```

Replace `YOUR_USERNAME` with your GitHub username. If Git says there is nothing to commit, that's okay; continue with the remote and push commands.

This pushes your current branch to your fork, leaving the original repository untouched.

Step 4 — Finish your lab submission

After testing the game, record the **after** video. Then upload these into the `Lab-4` folder of your lab repository:

- `before.mp4` — original gameplay
- `after.mp4` — fixed game with the new features
- Updated code
- Exported ChatGPT conversation or chat history PDF

Tell me what happens in Step 1: does the updated game run successfully? We'll handle any errors before you submit.

ds\63_anvil_dodge> git status
On branch main

```
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   game/anvil.py
        modified:   game/game_engine.py
        modified:   game/player.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        game/__pycache__/

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\nadhe\Downloads\63_anvil_dodge> git add game/player.py game/anvil.py
game/game_engine.py
warning: in the working copy of 'game/anvil.py', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'game/game_engine.py', LF will be replaced by CRLF the next time
Git touches it
warning: in the working copy of 'game/player.py', LF will be replaced by CRLF the next time Git
touches it
PS C:\Users\nadhe\Downloads\63_anvil_dodge> git commit -m "Fix boundaries and add gameplay
features"
[main e87e591] Fix boundaries and add gameplay features
 3 files changed, 266 insertions(+), 159 deletions(-)
PS C:\Users\nadhe\Downloads\63_anvil_dodge> git remote add myfork
```

https://github.com/YOUR_USERNAME/63_anvil_dodge.git
PS C:\Users\nadhe\Downloads\63_anvil_dodge> git push -u myfork HEAD
remote: Repository not found.
fatal: repository 'https://github.com/YOUR_USERNAME/63_anvil_dodge.git/' not found
PS C:\Users\nadhe\Downloads\63_anvil_dodge>
PS C:\Users\nadhe\Downloads\63_anvil_dodge> git remote add myfork
https://github.com/hemant-1215/63_anvil_dodge
error: remote myfork already exists.
PS C:\Users\nadhe\Downloads\63_anvil_dodge>

Bro, you're almost there! 😄 Your changes are safely committed locally (`e87e591`). The error happened because `YOUR_USERNAME` was a placeholder, and Git saved that incorrect URL as the `myfork` remote.

Let's fix it without losing your work.

Step 1 — Correct the remote URL

Your GitHub username is `hemant-1215` , so run:

```powershell
git remote set-url myfork https://github.com/hemant-1215/63_anvil_dodge.git
```

Step 2 — Check that your fork exists

Open your repository:

Open your GitHub repository ↗

If the page shows a 404, create the fork first using the original repository:

Open the original repository to fork ↗

Click **Fork** and select `hemant-1215`.

Step 3 — Push your committed changes

Once your fork exists, run:

```powershell
git push -u myfork HEAD
```

This uploads your current branch and your commit to your fork.

Step 4 — Verify

Refresh your GitHub repository page. You should see the updated `game/player.py`, `game/anvil.py`, and `game/game_engine.py` files.

The LF/CRLF messages are normal Windows line-ending warnings; they did not prevent your commit.

Do this now: run Step 1, then Step 3 if the fork already exists. If you get another error, paste it here and we'll sort it out.

give in a block

Copy and run these commands in PowerShell, one at a time:

```powershell
```

```
git remote set-url myfork https://github.com/hemant-1215/63_anvil_dodge.git

git push -u myfork HEAD
```

If you get **Repository not found**, create your fork first at the original repository:

https://github.com/SETAPESU26/63_anvil_dodge ↗

Then run the commands again.

is that all

Almost, bro! 🔥 Pushing the code is only part of Lab 4. Your handout requires a few more things before submission.

Lab 4 — Final submission checklist

Submission progress 3 of 8

- [x] ~~Clone the assigned repository~~
- [x] ~~Record 10-second before video~~
- [x] ~~Fix bug and implement three features~~
- [ ] Test the updated game
- [ ] Push updated code to your GitHub fork
- [ ] Record 10-second after video
- [ ] Export complete ChatGPT conversation as PDF